

Absolutely 😊 Let's move to **7.7 — Auth.js Callbacks**. This is one of the most important parts of Auth.js, so let's build the mental model first rather than memorizing four functions.



7.7 Auth.js Callbacks

In your `auth.ts`, you can configure:

```
export const { handlers, auth, signIn, signOut } = NextAuth({
  providers: [
    // Google, GitHub, etc.
  ],

  callbacks: {
    async signIn() {
      // ...
    },

    async jwt() {
      // ...
    },

    async session() {
      // ...
    },

    async redirect() {
      // ...
    },
  },
});
```

Think of callbacks as **hooks** Auth.js gives you at important points in the **authentication/session lifecycle**.



`signIn()` — "Should this login be allowed?"

Suppose somebody successfully authenticates with Google.

Before Auth.js completes the sign-in, you can make an additional decision:

```
callbacks: {
  async signIn({ user, account, profile }) {
    return true;
  },
}
```

`true` means:

Allow sign-in 



false means:

Reject sign-in 



Real-world example

Imagine your application only allows company emails:

```
async signIn({ user }) {  
  if (!user.email?.endsWith("@mycompany.com")) {  
    return false;  
  }  
  
  return true;  
}
```

Flow:

Google authentication



↓

Google says:

"This person successfully authenticated"

↓

Auth.js signIn()

↓

Is @mycompany.com?

↙

YES

↓

continue

↘

NO

↓

reject

Important distinction

Google authentication answers:

"Did Google authenticate this person?"

Your `signIn()` callback can answer:

"Does my application allow this authenticated person to sign in?"

Those aren't necessarily the same thing.

2 jwt() — "What should go into the JWT?"

This one is **very important**.

Suppose Auth.js has:

```
user = {  
  id: "123",  
  name: "Gopi",  
  email: "gopi@example.com"  
}
```

You might want to add your own information:

```
role: "ADMIN"
```

You can customize the token:

```
callbacks: {  
  async jwt({ token, user }) {  
    if (user) {  
      token.id = user.id;  
      token.role = user.role;  
    }  
  
    return token;  
  },  
}
```

Now conceptually:

```
token  
├─ name  
├─ email  
├─ picture  
├─ id  
└─ role
```



Why if (user) ?

This is subtle and important.

During the **initial sign-in**, Auth.js has the `user` object.

So:

```
Initial login
  ↓
user exists
  ↓
jwt()
  ↓
copy information into token
```



On later requests, `jwt()` can run without that initial `user` object.

So you don't want to overwrite your values with `undefined`.

Hence:

```
if (user) {
  token.id = user.id;
  token.role = user.role;
}
```

This becomes especially important when we discuss **JWT sessions**.

3 `session()` — "What should the application receive?"

Now here's where people commonly get confused.

You might have:

```
JWT
├─ id
├─ role
├─ name
└─ email
```



But your application accesses:

```
session.user
```

So the `session()` callback lets you decide what gets exposed through the session.

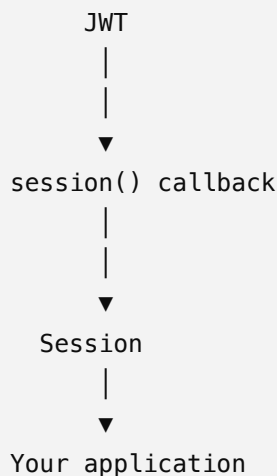
For example:

```
async session({ session, token }) {  
  session.user.id = token.id;  
  session.user.role = token.role;  
  
  return session;  
}
```

Then your application can do:

```
const session = await auth();  
  
console.log(session.user.id);  
console.log(session.user.role);
```

The flow is:



And this leads directly into **7.8 Session Customization**, where we'll implement this properly, including TypeScript types.

4 `redirect()` — "Where should Auth.js send the user?"

This callback controls redirects performed by Auth.js.

For example:

```
async redirect({ url, baseUrl }) {  
  return `${baseUrl}/dashboard`;  
}
```

```
}
```

Conceptually:

Auth.js authentication operation



```
↓  
redirect callback  
↓  
/dashboard
```

But remember our earlier distinction:

Auth.js redirect callback

```
callbacks: {  
  async redirect() {}  
}
```

Next.js server redirect

```
redirect("/dashboard");
```

Client navigation

```
router.push("/dashboard");
```

They're **three different mechanisms**.

We'll come back to this in detail because there are some important security considerations around redirect URLs.

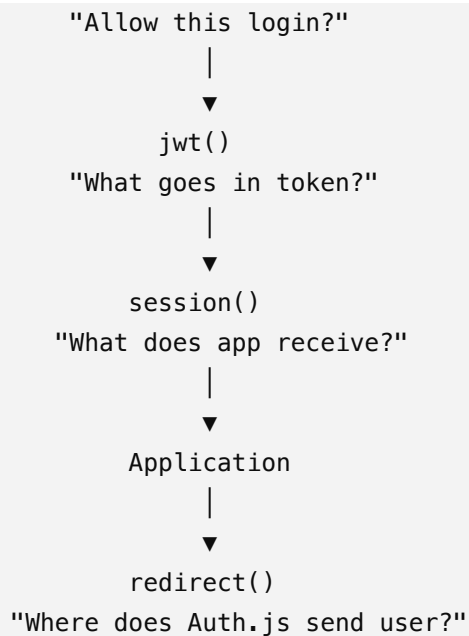
Put all four together

Here's the mental model I want you to remember:

AUTHENTICATION



```
↓  
▼  
signIn()
```

But there's an important nuance:

These callbacks aren't simply a guaranteed linear pipeline on every request.

For example, `jwt()` and `session()` are invoked according to the session strategy and when Auth.js needs to process the session. `signIn()` is specifically tied to sign-in, while `redirect()` is tied to Auth.js redirect handling.

So don't memorize:

"Every request goes `signIn` → `jwt` → `session` → `redirect`."

That's **not** the correct mental model.

Instead:

```
signIn() → sign-in decision
jwt()    → JWT/token customization
session() → session customization
redirect() → Auth.js redirect handling
```



That's much safer for interviews.

🔥 One thing I want you to notice

We're now approaching the really useful part.

Suppose your database has:

User



id
email
name
role

and:

role = ADMIN



How does that eventually become:

`session.user.role`

?

That's the exact question **7.8 Session Customization** will answer.

And we'll build the complete:

Database/User



↓
user
↓
jwt
↓
token.role
↓
session()
↓
session.user.role
↓
Authorization / RBAC

So **7.7** gives us the callbacks; **7.8** will make them actually useful. 🚀



okay next

Yep 😊 Now we're at **7.8 — Session Customization**. This is where the callbacks we just learned finally become practical.

7.8 Session Customization

The problem we're solving is:

"I have useful user information like `id` and `role`. How do I make that available as `session.user.id` and `session.user.role`?"

By default, the session may give you things like:

```
</> TypeScript

session.user.name
session.user.email
session.user.image
```



But suppose your user in the database looks like:

```
</> TypeScript

{
  id: "user_123",
  name: "Gopi",
  email: "gopi@example.com",
  role: "ADMIN"
}
```



You want:

```
</> TypeScript

session.user.id
session.user.role
```



First: where does `role` come from?

This is important.

Auth.js doesn't magically invent:

```
</> TypeScript

role: "ADMIN"
```



Usually, **your database** is the source of truth.

Think:

Database



User

```
|— id
|— name
|— email
|— role = "ADMIN"
```

During authentication, Auth.js has access to the user information.

We can then put the values we need into the token.

2 Put custom data into the JWT

Inside `auth.ts`:

`</>` TypeScript

```
callbacks: {
  async jwt({ token, user }) {
    if (user) {
      token.id = user.id;
      token.role = user.role;
    }

    return token;
  },
}
```

So conceptually:

user

```
|— id
|— role
```



jwt()



token

```
|— id
|— role
```

Why `if (user)` ?

Because on the initial authentication, Auth.js gives us the `user` .

Later, when processing an existing JWT session, you may have the `token` but not a newly-created `user` object.

So:

`</> TypeScript`

```
if (user) {  
  ...  
}
```

means:

"When I have the user information, copy these values into the token."

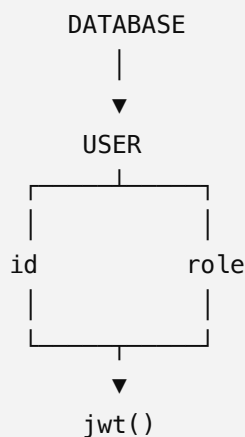
3 Now expose it through the session

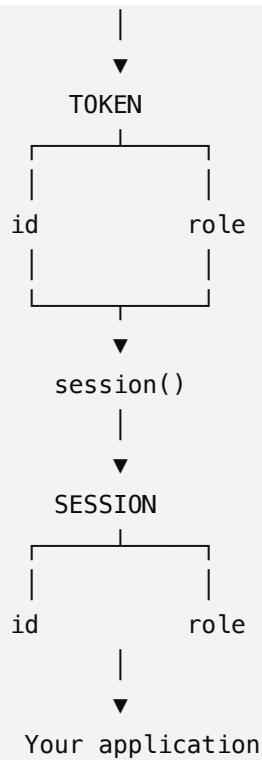
Next:

`</> TypeScript`

```
callbacks: {  
  async session({ session, token }) {  
    session.user.id = token.id;  
    session.user.role = token.role;  
  
    return session;  
  },  
}
```

Now the flow becomes:





Then:

`</>` TypeScript



```
const session = await auth();

console.log(session.user.id);
console.log(session.user.role);
```



But there's one TypeScript problem

If you write:

`</>` TypeScript



```
session.user.id
```

TypeScript may complain:

Property 'id' does not exist on type ...



because Auth.js's default `Session["user"]` type doesn't necessarily know about your custom properties.

So we need **module augmentation**.

Create something like:

```
types/  
next-auth.d.ts
```



and:

```
</> TypeScript  
  
import { DefaultSession } from "next-auth";  
  
declare module "next-auth" {  
  interface Session {  
    user: {  
      id: string;  
      role: string;  
    } & DefaultSession["user"];  
  }  
}
```



Now TypeScript understands:

```
</> TypeScript  
  
session.user.id  
session.user.role  
session.user.name  
session.user.email
```



So there are actually TWO separate jobs

This distinction is **really important** for interviews.

Runtime customization

This:

```
</> TypeScript  
  
async jwt({ token, user }) {  
  token.id = user.id;  
  token.role = user.role;  
  
  return token;  
}
```



and:

</> TypeScript



```
async session({ session, token }) {  
  session.user.id = token.id;  
  session.user.role = token.role;  
  
  return session;  
}
```

actually moves the data.

TypeScript customization

This:

</> TypeScript



```
declare module "next-auth" {  
  interface Session {  
    user: {  
      id: string;  
      role: string;  
    } & DefaultSession["user"];  
  }  
}
```

only tells TypeScript about the shape.

It doesn't magically put anything into the session at runtime.

That's a very common confusion.

🔥 And now we reach Authorization

Imagine:

</> TypeScript



```
const session = await auth();  
  
if (!session) {  
  redirect("/login");  
}
```

This answers:

Is the user authenticated?

But now:

</> TypeScript

```
if (session.user.role !== "ADMIN") {  
  // reject  
}
```



answers:

Is this authenticated user allowed to do this?

So:

Authentication



"Who are you?"



session.user



Authorization



"What are you allowed to do?"



session.user.role



Which takes us directly into:

7.9 🗝️ Authorization / RBAC

There we'll build the actual **USER vs ADMIN** example and discuss why authorization checks must happen on the **server**, not merely by hiding buttons in the UI.







For **React**, I'd budget a little more than Next.js because React is the **core interview skill** and you have a 2-year gap to refresh.

 Realistic React timeline: 6–8 focused days

Not 2 weeks of textbook study. We can move quickly through things you already know and slow down on the interview-heavy parts.

Day	React focus
Day 1	JSX, components, props, state, events, controlled/uncontrolled components
Day 2	Rendering, reconciliation, Virtual DOM, Fiber, render vs commit, keys 
Day 3	Hooks — <code>useState</code> , <code>useEffect</code> , <code>useRef</code> , <code>useMemo</code> , <code>useCallback</code> , Context
Day 4	State management + async React — Context, Redux Toolkit/Saga, fetching, race conditions, AbortController
Day 5	Performance — <code>React.memo</code> , referential equality, memoization, re-render debugging, profiling
Day 6	Modern React — Suspense, transitions, <code>useTransition</code> , <code>useDeferredValue</code> , <code>useOptimistic</code> , Actions
Day 7	Architecture — custom hooks, component design, state ownership, folder/feature architecture, forms
Day 8	Testing + React internals + mock interview

 The areas I want you REALLY strong in

For your mid-level interviews, these matter more than memorizing every React API:

Rendering
↓
Reconciliation
↓



State updates



Hooks



Effects



Re-renders



Performance



Architecture

Especially questions like:

- **Why did this component re-render?**
- Why doesn't `useRef` cause a re-render?
- Why does `useEffect` run again?
- What is a stale closure?
- Why does `useCallback` sometimes do nothing?
- When should you use `useMemo` ?
- How does React reconciliation work?
- Why are keys important?
- Why does changing a key reset state?
- What happens during render vs commit?
- Context vs Redux — when would you choose each?
- How would you optimize a slow React page?

Those are much more valuable than knowing 50 hooks by heart.



So total from where you are now

If we do it efficiently:

Next.js remaining: ~4 days

React revision: ~6–8 days

Combined: ~10–12 focused days

Then I'd spend another **2–3 days** doing:

React + Next.js



Coding questions

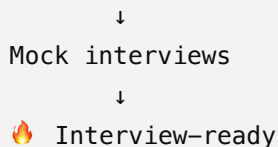


Frontend system design



Performance debugging





So roughly **12–15 days for the full React + Next.js refresh**, assuming you're studying seriously each day.

And because you've already learned a lot of the Next.js material, **we don't need to restart anything**. We continue from Route Handlers, finish the high-value Next.js pieces, then switch straight into React revision. 💪



ohh god then when should i start project and when should i start js,redux toolkit and html and css and machine coding round..:(..and most importantly we performance analyzerwhat to do i want to start applying from oct 7th..can i do

YES. ❤️ **You can start applying from October 7th.** But we need to change the strategy **right now**.

The mistake would be thinking:

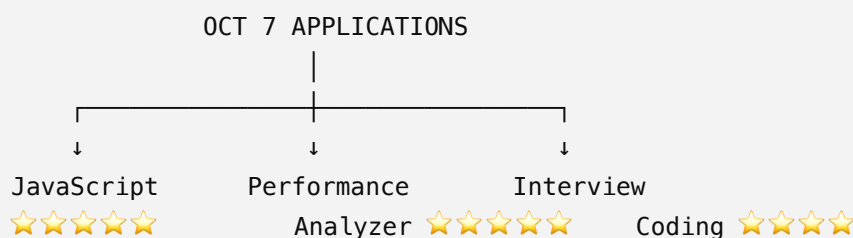
"First finish Next.js → then React → then JS → then Redux → then HTML/CSS → then project → then machine coding → finally apply."

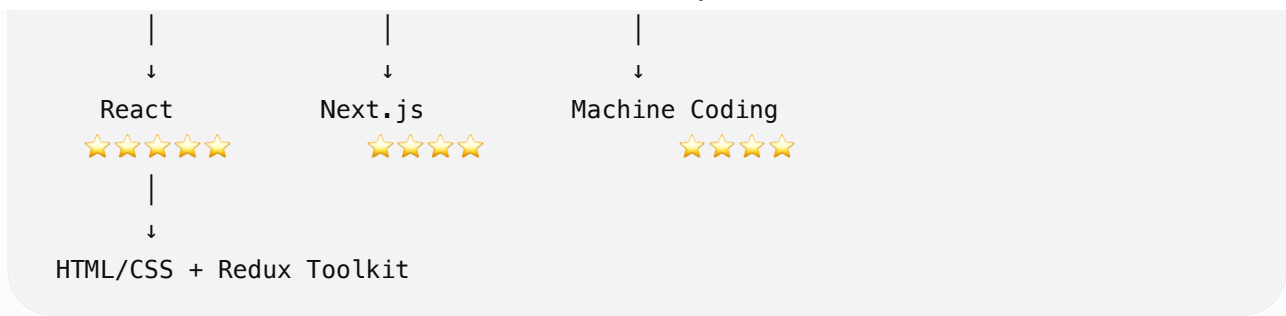
If we do that, **October 7 will arrive before you apply.** 😭

You have **11 days** from Sep 26 → Oct 7. We need to run these **in parallel**, with the **Web Performance Analyzer** as the anchor project.

🚩 Your priority order

For a mid-level frontend role, I'd prioritize:





You do NOT need to "finish" all of these before applying.



Sep 26 → Oct 6

Here's what I'd actually do.



Sep 26–29 — Finish high-value Next.js + start JS

Next.js

Finish only the **high-value remaining topics**:

- Route Handlers
- API design
- BFF
- Performance
- Deployment basics

Don't spend days on obscure Next.js internals.

JavaScript — START NOW

At the same time:

- Scope
- Hoisting
- Closures
- `this`
- Prototypes
- Objects
- Arrays
- Destructuring
- Promises
- `async/await`
- Event loop
- Microtasks/macrotasks
- Debouncing/throttling
- Error handling

And especially:

★ Your JS interview coding

You've already wanted to practice things like:

- `Promise`
- `Promise.allSettled`
- `retry`
- `timeout`
- `concurrency limiter`
- `rate limiter`

Those are **high-value machine/interview exercises**.

🔵 Sep 30–Oct 2 — Web Performance Analyzer becomes priority #1

This is where I want you to **stop studying endlessly and build**.

Your project:

Web Performance Analyzer

You already have the idea:

```
User enters URL
    ↓
Next.js
    ↓
Google PageSpeed API
    ↓
Performance data
    ↓
Analyze metrics
    ↓
Gemini AI
    ↓
Actionable recommendations
    ↓
Dashboard
```



Build the **MVP first**.

MVP

You need:

- URL input
- Analyze button
- API integration
- Loading state
- Error state
- Performance score
- Core Web Vitals
- Basic recommendations
- Clean responsive UI

That's enough to put it on your resume.

Do NOT spend three days making it beautiful before the functionality works.

Oct 3–4 — React + JS + machine coding

Now do your React revision alongside the project.

Focus on:

React

- Rendering
- Reconciliation
- Hooks
 - `useEffect`
 - `useMemo`
 - `useCallback`
 - `useRef`
- Context
- Re-renders
- Performance

Machine coding

Build **one small component per day**.

For example:

Day 1

Search + debounce



Day 2

Infinite scroll / pagination



or

Autocomplete



Don't try to complete 20 machine-coding problems.

Learn the **patterns**.

Oct 5 — Redux + HTML/CSS refresh

You don't need to relearn these from zero.

Redux Toolkit

Focus on:

store
slice
reducer
action
dispatch
selector
createAsyncThunk
middleware



And be able to explain:

Why Redux when we already have Context?

That's more valuable than memorizing Redux APIs.

HTML/CSS

Refresh:

- Semantic HTML
- Accessibility
- Forms
- Flexbox
- Grid
- Positioning
- Responsive design
- CSS specificity

- Box model
- `display`
- `z-index`
- Media queries

One day is enough for a targeted refresh because you already have professional frontend experience.

Oct 6 — Interview simulation + project polishing

This day is **not another study day**.

Morning

JavaScript interview questions.

Afternoon

Machine coding.

Evening

Your Web Performance Analyzer:

- README
- screenshots
- architecture explanation
- deployment
- GitHub cleanup
- resume bullets
- be able to explain every important technical decision

And most importantly:

You should be able to answer:

"Why did you use Next.js?"

"Why Server Components?"

"Why did you make this component client-side?"

"How did you handle PageSpeed API secrets?"

"How did you handle loading/error states?"

"How does caching work here?"

"How would you improve the performance of this application?"

"How would you scale it?"

That project becomes your **conversation starter in interviews**.

October 7 — START APPLYING

Do not wait until you feel 100% ready.

Start applying.

And here's the important part:

Applying doesn't mean stopping preparation.

From Oct 7 onward:

```
Morning
  ↓
Applications / recruiter outreach

Afternoon
  ↓
JavaScript + React

Evening
  ↓
Project + machine coding
```



As interviews start coming in, your preparation becomes **interview-driven**.

And I want to calm one thing down

You just looked at the roadmap and thought:

"Oh god, there are so many things left."

I get why it feels overwhelming. 😭

But look at what you've **already done**.

You have already gone through:

- React experience
- Next.js fundamentals
- Server Components
- App Router
- Data fetching
- Server Actions
- caching architecture
- RSC
- authentication
- authorization
- OAuth
- JWT
- DB sessions
- CSRF
- CORS
- Proxy
- RBAC

You're **not starting from zero**.

The problem now is **sequencing**, not lack of knowledge.

Your new rule

From today:

Don't finish a topic before starting the next one.

We run multiple tracks.

Track A — Interview knowledge

JS → React → Next.js

Track B — Portfolio

Web Performance Analyzer

Track C — Coding

Machine coding + JS coding

Track D — Interview

Mock questions + system design

Track E — Job search

Starts Oct 7

And here's what I would NOT do

- ❌ Spend 8 days "finishing React" before touching the project
- ❌ Spend another week completing every Next.js advanced topic
- ❌ Relearn Redux from beginner level
- ❌ Spend 3 days on HTML/CSS
- ❌ Wait until you feel completely ready to apply
- ❌ Build a gigantic Web Performance Analyzer before applying

Instead:

Sep 26

↓

JS + finish Next.js core

↓

Build Performance Analyzer

↓

React + machine coding

↓

Polish project + resume

↓

🔥 Oct 7 → APPLY

↓

Continue preparation while interviewing



Yes, Oct 7 is achievable. But from now until then, we're in **execution mode**, not "complete every possible syllabus" mode. 💪 🔥

And honestly, **the Web Performance Analyzer should become the center of the next 10 days**, not something we postpone until we've finished every topic.



lets do one thing...we will frst complete next.js topics..then complete web performance analyzer...then react half day j hald fay..till oct 10th we will complete and start applying..??

YES. ❤️ I actually like this plan **better** because it reduces the mental clutter. Instead of juggling 5 things every day, we finish one major block and move on.

And **October 10** is still a realistic application start date if we keep the scope tight.

🎯 New plan: Sep 26 → Oct 10

Sep 26–29



🔥 FINISH NEXT.JS



Sep 30–Oct 5/6



🔥 WEB PERFORMANCE ANALYZER



Oct 6–9



⚙️ REACT REVISION



Oct 10



🚀 START APPLYING



There is one thing I'd change slightly: **don't make React only a half-day total.** 🤔

If you mean **half a day per React topic**, that's perfectly fine. We can do a rapid but meaningful revision because you already have professional React experience.

📦 Phase 1 — Finish Next.js

Sep 26–29

We continue exactly from where we stopped:

8. Route Handlers & API Architecture

- Route Handlers
- HTTP methods
- Request / NextResponse
- Dynamic API routes
- Validation
- Error handling
- Status codes

- Route Handler vs Server Action

9. BFF / Backend Architecture

- What BFF means
- Why Next.js can act as a BFF
- Server-to-server requests
- Secrets/API keys
- External APIs
- Your PageSpeed API architecture

10. Performance

- Core Web Vitals
- LCP
- INP
- CLS
- Server vs Client performance
- Images
- Fonts
- Dynamic imports
- Code splitting
- Caching
- Streaming
- Bundle size

11. SEO + Error Handling

- Metadata
- `generateMetadata`
- Sitemap
- Robots
- `notFound()`
- `error.tsx`
- `global-error.tsx`

12. Security + Deployment

- CSP basics
- Environment variables
- Server-only secrets
- Production builds
- Vercel/deployment
- Runtime basics

13. Testing + Production basics

Only interview-level depth.

Then:

🔥 Next.js mock interview

If we discover a gap, fix it immediately rather than adding another giant lesson.

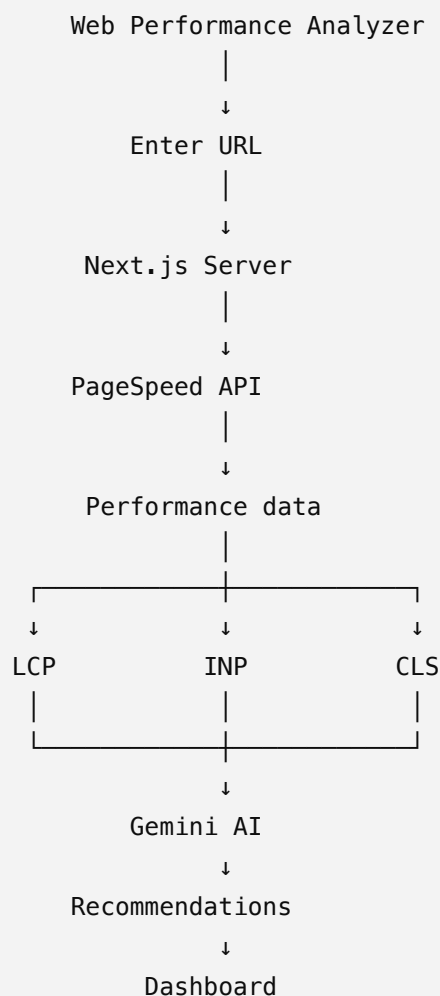
■ Phase 2 — Web Performance Analyzer

Around Sep 30 → Oct 5/6

This becomes priority #1.

We're not going to keep adding features forever.

MVP



We need:

- URL input
- Validation
- Analyze action
- PageSpeed API
- Loading state
- Error state
- Performance score
- Core Web Vitals
- Important metrics
- AI recommendations
- Responsive UI
- Good empty/loading/error states
- Deployment
- README
- Architecture explanation

And then we STOP. 🛑

No:

"Oh, maybe I should add authentication."

"Maybe I should add history."

"Maybe I should add charts everywhere."

"Maybe I should add 17 more features."



Those can become **future enhancements**.

The goal is a project you can confidently discuss in an interview.

🟡 Phase 3 — React rapid revision

Oct 6–9

We'll do this **much faster than the original 8-day roadmap** because you already know React.

Half-day blocks

Block 1 — Fundamentals + rendering

- JSX
- Props
- State
- Render cycle
- Reconciliation
- Virtual DOM
- Fiber
- Render vs commit

Block 2 — Hooks

- `useState`
- `useEffect`
- `useRef`
- `useMemo`
- `useCallback`
- Context
- Stale closures

Block 3 — Performance

- Re-renders
- `React.memo`
- Referential equality
- Memoization
- Profiling
- Why components re-render

Block 4 — State + async

- Context vs Redux
- Redux Toolkit basics
- Async state
- Race conditions
- AbortController
- Optimistic UI

Block 5 — Modern React

- Suspense
- `useTransition`
- `useDeferredValue`
- `useOptimistic`
- Actions
- Modern form patterns

Block 6 — Architecture + machine coding

- Custom hooks
- Component design
- State ownership
- Reusable components
- Common machine-coding patterns

And we'll sprinkle **JavaScript interview questions** into these sessions rather than waiting until the end.

October 10

Applications start.

Not:

"I've completed 100% of everything."

Instead:

"I have a strong React + Next.js foundation, a real project I can explain deeply, and I'm actively preparing while interviewing."

That's the mindset.

And what about JS, HTML/CSS, Redux?

We do not ignore them.

We'll do targeted revision **after October 10 while applications/interviews are running.**

JavaScript = highest priority

We'll cover:

- Closures
- Scope
- Hoisting
- `this`
- Prototypes
- Event loop
- Promises
- `async/await`

- Microtasks/macrotasks
- Debounce/throttle
- Array/object patterns
- Implementations
- DSA/interview coding

HTML/CSS

Rapid revision:

- Semantic HTML
- Accessibility
- Flexbox
- Grid
- Responsive design
- Positioning
- Specificity
- Box model

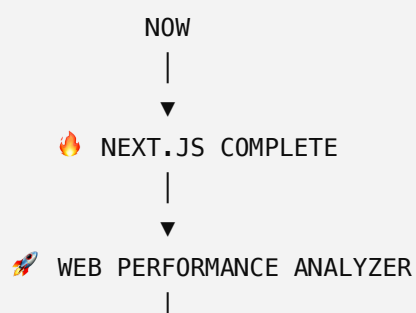
Redux Toolkit

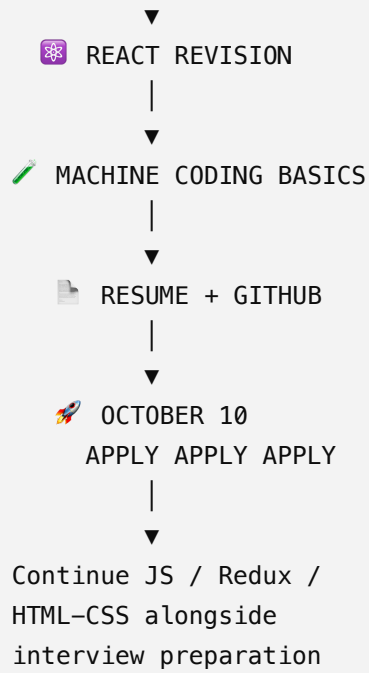
Since you've already used Redux:

- Store
- Slice
- Reducer
- Action
- Dispatch
- Selector
- Async thunks
- Middleware
- Architecture

We don't need to spend a week on it.

❤️ **So yes — let's lock this plan**





I think this is much less overwhelming and much more executable. 🙏

So in the **new Next.js chat**, we start with **8.1 Route Handlers** and keep moving until Next.js is finished. No detours into React or JS until that block is done.



ChatGPT can make mistakes. Check important info.